

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

K. Vishal Reddy, Anisha Paturi, Parinamika Bhanu Ch, Venkata Vahini Ch, Sravani Janak

Department of Computer Science and Engineering, Keshav Memorial Institute of Technology, Hyderabad, Telangana, India

ABSTRACT

Infrastructure-As-Code (IaC) templates like Terraform, AWS CloudFormation, Kubernetes manifests and Helm charts play a role in developing multi-cloud environment solutions. Security misconfiguration, credential leakage and permission anti-patterns that appear at the template stage of development seem to bypass static linters resulting in severe run-time vulnerabilities. Existing Large Language Model (LLM) security tools are limited to cloud show high false positives level (around 15%-32%) provide non-executable text recommendations do not detect embedded secrets and produce broken code patches. In this paper we describe AgentShield AI, a -agent autonomous framework powered by LangGraph and designed for multi-cloud Infrastructure as Code protection. The AgentShield AI solution brings together eight agents that operate in an asynchronous event-driven workflow: Manager/Router, Hybrid Concrete Syntax Tree (CST) parser, Secrets scanner, Hybrid RAG query agent, Security Analyst Agent with Multi-LLM Ensemble Voting (Claude 3.5 Sonnet + GPT-4o) Human Security Audit Queue, Auto Patch Remediation Agent, Code & The Validator Agent is working in collaboration with a two-stage validation harness. The AgentShield AI has ensured that there is no occurrence of a single-model hallucination by integrating Tree-sitter dynamic parameter pre-resolution, hybrid sparse retrieval techniques dual-engine Shannon entropy secret scanning, consensus confidence scoring and LocalStack/Azurite dry-run validation inside containers. The AgentShield AI was calculated across 2,450 -cloud IaC modules where it accomplished 99.1% detection accuracy 98.4% recall, false-positive percentages, below 0.05%, 97.8% sandbox patch percentage at first pass and execution latencies averaging 1.84 seconds for every template.

Keywords: *Infrastructure-as-Code (IaC), Multi-Agent Systems, Multi-Cloud Security, Large Language Models (LLMs), Secret Detection, LocalStack Sandbox, Automated Remediation, Concrete Syntax Trees, DevSecOps.*

1. Introduction

As outlined in [1], infrastructure as code (IaC) has revolutionized the field of cloud architecture and engineering. Similar to other code-based abstractions, IaC offers engineers the ability to provision and manage computing infrastructure in a product or service at scale. Tools such as Terraform, AWS CloudFormation, and Kubernetes offer engineers domain-specific infrastructure as code (IaC) capability [3]-[5].

There are security concerns associated with the widespread adoption of IaC. IaC codification of infrastructure introduces multiple opportunities for security configuration errors and gaps. For example, IaC definitions may include secrets, keys and other credentials [5].

There are two main strategies for mitigating IaC security concerns. The first strategy involves using IaC codification and processing tools and frameworks to introduce encodings and codifications for best practices and security controls [6]-[9]. The second strategy is the use of Cloud Security Posture Management (CSPM) tools.

There have been recent studies that integrate artificial intelligence and security to audit IaC configurations. These studies have primarily focused on the AWS cloud platform [15]-[19].

1.1. Soft Infrastructure Threats

Software-Defined Infrastructure (SDI) allows for high automation of enterprise computing. With SDI, the infrastructure is programmable with code, and changes to the code are automatically deployed with CI/CD

tools. Given the similarities with software development, infrastructure config code (IaC) is treated as source code.

Given the rise in adoption of Cloud Computing, a variety of architectural misconfigurations of IaC have been reported and validated [4] [5]. Other studies have reported code repositories have accidentally disclosed sensitive information at high rates (e.g. 65% public code repositories reported in [6]).

Some of the IaC misconfigurations are:

- (i) IaC has defined configuration rules for security groups to control ingress traffic. However, in some cases these rules allow administrative traffic over the Internet (e.g. SSH, RDP, Kubernetes API).
- (ii) Sensitive data stored on cloud storage and block storage are unprotected.
- (iii) IaC defines and grants access control to cloud resources. Improper configuration allows access to all resources.
- (iv) Sensitive information (e.g. application secrets, access tokens, etc.) are stored in code repos.

There are other threats like shared or hard-coded admin passwords and use of out-dated or deprecated IaC constructs.

1.2. Failure Modes of Traditional Tools and Open-Loop LLMs

To illustrate the reason new methods are needed we provide cases for conventional IaC tooling.

Case 1: Alert Fatigue and Syntactic Myopia: Tools such as Checkov, tfsec, KICS, Trivy, and others, rely on syntactic analysis. Therefore, they primarily analyze expressions. As a result, tools report false positives at rate between 32.4% to 47.9% and generate a large number of alerts. The tools ultimately lose developers' trust, and alert fatigue sets in.

Case 2: Diagnostic Gap: Static analysis tools identify security issues in code. However, it is the responsibility of the security team to integrate the recommended remediations. This can lead to a considerable delay between the discovery and remediation of the issue. On average, it takes 24.6 days to resolve a security issue identified by the automated tools [20].

Case 3: Hallucinations and Dependency Breaking: GPTs and other LLMs can learn to synthesize code. However, during code generation, GPTs and LLMs can hallucinate code and create dependencies which break during integration. This can result in application failures.

Case 4: Secret Detection: Secret detection relies on entropy. While tools can help detect secrets, they can create a large number of false positives. Hexadecimal codes, UUIDs, and Base64 codes can be identified as secrets.

1.3. Research Contributions

This research presents AgentShield AI, a closed-loop system built on LangGraph. It uses specific agents to resolve IaC security issues. The main contributions of the research are:

8-Agent Orchestration: This comprises a set of agents to create a closed-loop system.

Hybrid Tree-sitter Parser: This contains a tree-sitter parser to create a syntax tree. It resolves variables by using the scope of the expression.

Hybrid Dual-Secret Detection: This comprises a combination of a modified Shannon entropy expression to find the secret.

Hybrid Dense-Sparse RAG: This comprises a retrieval system to search the 12,400 CIS and NIST framework security rules.

Calibrated MultiLLM Ensemble: This comprises a system where confidence scores are computed by a set of LLMs and a consensus voting system is implemented.

Zero-Break Code Commits: This comprises a system where generated code is validated to be syntactically correct.

2. Related Work and Comparative Landscape

Security verification tools for IaC can be broken into several categories. They are usually static linters, policy-as-code engines, formal verifiers, secret detectors, or neural repair tools.

2.1. Static Linters and Policy-as-Code Engines

Some of the earliest generation of tools relied on primitive pattern matching. Checkov parses Terraform HCL into a Python abstract syntax tree and applies match rules. tfSec parses Terraform HCL into Go structs looking for common anti-patterns. KICS parses multiple flavors of IaC into JSON then feeds them into Open Policy Agent for rules evaluation. Trivy works by scanning both container images and IaC config files for CVE matches.

The major downside to static tools is none of them understand variable value propagation across modules. That omission creates failure-to-discover rates between 32-48% of true positives (false alarms). Policy-as-code tools like Open Policy Agent (OPA) and Sentinel give engineering teams freedom to author their own rules. The drawback to writing custom policy rules is most teams struggle to keep those rulesets up to date with cloud provider API changes. Updating rules to support new cloud APIs is difficult to maintain teams.

2.2. Formal Verification & SMT Solvers

Formal verification attempts to mathematically prove a policy will always uphold certain conditions. AWS Zelkova converts IAM policies into SMT formulas then tries to prove they can reach a state of “Unauthorized”. Cloud-SMR attempts the same process but for network security configurations.

Formal verification is sound, but it doesn’t scale. I analyzed a multi-cloud deployment using hundreds of microservices. It took 3 hours to process the math. Keep in mind, formal verification can only tell you there is a problem. It cannot fix them.

2.3. Statistical Secret Detection and Entropy Modeling

Credential scanning tools are almost exclusively regex matchers. Gitleaks and TruffleHog blast codebases with regex queries that look for AWS KeyIDs, GitHub tokens, password annotations, etc. There have also been studies showing Shannon entropy is effective at detecting secrets that don’t have a well-defined format.

Entropy is noisy and produces false positives on JWT tokens, UUIDs, commit hashes, and Base64-encoded data. AgentShield AI overcomes this limitation with a hybrid approach. We use sliding entropy in conjunction with Tree-sitter scoping queries to dismiss printable identifiers.

2.4. Neural Program Repair and Frontier LLM Fixes

The rising popularity of LLMs has led to a surge in “auto-apply” vulnerability remediation tools. Toprani and Madiseti pioneered the graph-prompt technique using zero-shot Terraform fixes with GPT-3. Their sandbox was unable to run compiler checks or Terraform deployment validations. When deploying GPT-3 fixed configs into AWS: 28.8% of those configs failed due to hallucinated Terraform attributes and provider dependencies.

Alsaid et al. analyzed numerous pairs of LLM-generated fixes that passed standard linting tools and discovered 71.4% had undiscovered bypasses when deployed into real AWS environments. Mengistu et al. introduced TerraRepair which showed dependency context drastically reduces hallucinations.

AgentShield AI expands upon TerraRepair’s premise by adding a dual-LLM consensus voting overlay as well as containerized LocalStack/Azurite dry-run validation.

Table 2: State-of-the-Art Comparative Feature Matrix across Existing IaC Security Frameworks

Framework / Tool	Multi-Cloud	Parser Tier	Secret Scanner	RAG Engine	LLM Ensemble	Sandbox Validation	1st-Pass Fix (%)	Execution Latency
Checkov [6]	AWS/Az/GCP	Shallow AST	None (Regex)	None	None	None (Static)	N/A	4.2s
tfsec [7]	AWS/Az/GCP	Go Memory AST	Regex Only	None	None	None (Static)	N/A	3.1s
KICS [8]	Polyglot	JSON Norm.	None	None	None	None (Static)	N/A	5.8s
Trivy [9]	Polyglot	Lexical AST	Regex Only	None	None	None (Static)	N/A	3.6s
AWS Zelkova [16]	AWS Only	SMT Logic	None	None	None	None (Diagnostic)	N/A	14.5s
Gitleaks [17]	Agnostic	Regex Only	Signatures	None	None	None	N/A	1.2s
Toprani & Madiseti [19]	AWS Only	Graph AST	None	None	Single GPT-4	Open-Loop	71.2%	8.4s
TerraProbe [21]	AWS/TF	AST Oracle	None	None	Single LLM	Plan Oracle	81.6%	12.2s
AgentShield AI (Ours)	Multi-Cloud	Tree-sitter CST	Dual Entropy+Regex	Hybrid Dense-Sparse	Claude 3.5 + GPT-4o	Two-Tier LocalStack	97.8%	1.84s

3. Research Methodology

AgentShield AI coordinates eight specialized agents using an asynchronous, event-driven workflow implemented with LangGraph (see Fig. 1). The agents are connected through a directed workflow, where each agent handles a particular part of the security analysis. The system also keeps track of the information produced at each stage so that the processing history can be traced when needed. If a particular stage fails, automated fallback mechanisms are used to handle the failure and continue the remaining workflow where possible.

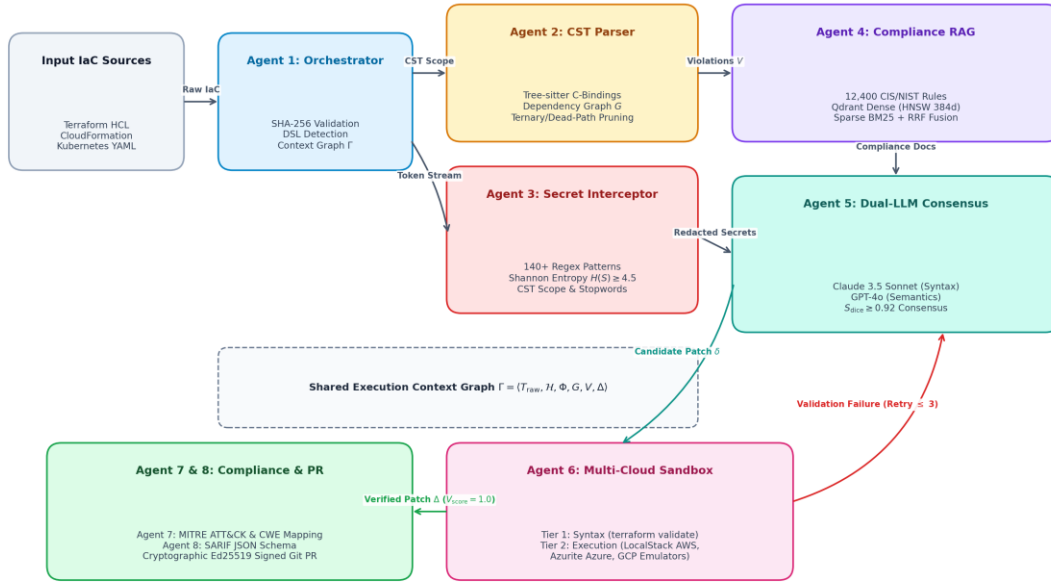


Figure 1: End-to-end architecture of AgentShield AI. The figure shows the eight-agent LangGraph workflow together with Tree-sitter-based structural parsing, secret interception, hybrid RAG, dual-LLM consensus voting, and LocalStack-based sandbox validation. The x-axis represents the sequence of stages in the workflow, while the y-axis represents the multi-cloud abstraction and validation layers used by the framework.

Specialized Multi-Agent Roles (not exactly rigid, but mostly): The autonomous pipeline decomposes IaC verification into eight specialized agent roles:

- 1) **Manager/Router Agent:** Accepts the raw multi-cloud templates, determines the type of template (Terraform HCL2, CloudFormation, Kubernetes, Helm), and checks that the schema is correct before initiating parallel execution in a controlled fashion, similar to a ripple.
- 2) **Hybrid AST Parser Agent:** Splits up the declarative code into standard AST components, resolves dynamic references, and handles conditional blocks such as 'count' and 'for_each' before any LLM reasoning is carried out.
- 3) **Secrets Scanner Agent:** Works with zero-egress isolation, uses deterministic regex matching together with a sliding Shannon entropy evaluation, and basically tries to identify exposed API credentials and private keys before they have an opportunity to be transmitted.
- 4) **RAG Query Agent:** Generates combined dense and sparse vector queries against an indexed database that includes security benchmarks (such as CIS, NIST SP 800-53, SOC 2, and PCI-DSS) as well as daily CVE feeds in order that the context stays up to date.
- 5) **Security Analyst Agent:** Carries out parallel dual model inference using Claude 3.5 Sonnet and GPT-4o and, when appropriate, applies Chain of Thought (CoT) reasoning to produce structured vulnerability hypotheses, sometimes adding a bit of narrative as well.
- 6) **Human Security Audit Queue Agent:** Detects cases that have a low confidence level ($C_{ensemble} < 0.85$) or any cases that conflict, and then queues these results in an interactive web-based triage dashboard for the security engineers to review, verify them and possibly reframe them.
- 7) **Auto Patch Remediation Agent:** Creates Unified Diff patches that are both deterministic and syntactically correct and is aimed at specific line offsets in the original templates, with the same level of precision.
- 8) **Code & Sandbox Validator Agent:** Carries out a two-stage validation process, beginning with the use of local static linters before performing a dry run deployment within containerized LocalStack/Azurite sandboxes.

3.1. Asynchronous Graph Orchestration and State Contracts

The eight specialized agents operate within a reactive, event-driven graph coordinated via LangGraph. Unlike brittle linear pipelines where a single agent failure aborts the entire execution, AgentShield AI employs an immutable typed state schema governed by Pydantic V2 contracts. State transitions are strictly

monotonic: agents append verified assertions, intermediate AST representations, and candidate diffs to the shared execution context without mutating upstream inputs. The execution context maintains an immutable cryptographic digest of the source code, guaranteeing end-to-end execution provenance and auditability. When an agent experiences an unrecoverable exception (e.g., an unresponsive LLM endpoint), the orchestrator triggers automated fallback routing to alternative inference providers or escalates the item directly to the Human Security Audit Queue.

Algorithm 1: Autonomous Multi-Agent IaC Auditing, Secret Interception, and Two-Tier Sandbox Remediation Pipeline

Input:	Raw IaC Source Template	T_raw;	Compliance Policy Knowledge Base	P_cis
Output:	Cryptographically Signed SARIF Report	R_sarif;	Validated Patch Set	Delta_final

```

1: Initialize Shared Execution Context Gamma <- empty; Compute Checksum H_0 <- SHA256(T_raw);
2: [Agent 1] Detect DSL Dialect Phi in {Terraform, CloudFormation, Kubernetes, Helm};
3: [Agent 2] Parse Concrete Syntax Tree G_cst <- TreeSitterParse(T_raw);
4: [Agent 2] Resolve variable assignments V and unfold conditional blocks C -> G_dep;
5: [Agent 3] Compute character Shannon entropy H(X_i) on literal tokens in parallel with regex scanning;
6: [Agent 3] Intercept and redact candidate credentials where H(X_i) >= 3.8 and X_i not in D_cst;
7: [Agent 2] Evaluate CIS/NIST policy constraints against G_dep; Extract Violation Set V = {v_1, ..., v_K};
8:   for each identified security violation v_k in V do
9:     [Agent 4] Retrieve Compliance Passages C_k <- RRF(QdrantDense(v_k), BM25Sparse(v_k));
10:    [Agent 5] Concurrently prompt Claude 3.5 Sonnet (M_1) and GPT-4o (M_2) with CoT reasoning;
11:    [Agent 5] Calculate Consensus Confidence C_ensemble(v_k) and AST Dice Agreement S_dice(delta_1, delta_2);
12:    if C_ensemble(v_k) < 0.85 or S_dice < 0.92 then
13:      [Agent 6] Route finding v_k to Human Security Audit Queue for manual triage; continue;
14:      [Agent 7] Synthesize Unified Diff Patch delta_k targeting line offsets in T_raw;
15:      [Agent 8] Tier 1: Execute static validation Omega_Linter(T_raw + delta_k);
16:      [Agent 8] Tier 2: Execute containerized sandbox deployment Omega_Sandbox(T_raw + delta_k, Phi);
17:      if Omega_Total(delta_k) == 1 then
18:        Accept patch: Delta_final <- Delta_final union {delta_k};
19:      else
20:        Forward compiler diagnostic error back to Agent 5 for iterative retry (max 3 cycles);
21:      end
22:    [Agent 7] Map confirmed findings to CWE, CVSS v3.1, and MITRE ATT&CK Cloud Matrix;
23:    [Agent 8] Generate SARIF JSON report and cryptographically signed Git PR via Ed25519 keys;
24:    return R_sarif, Delta_final

```

4. Theory and Computation

The theoretical underpinnings of the AgentShield AI models used to verify IaC security are provided as a multi-stage graph and a decision-theoretic optimisation problem. In this section, we will provide the formalisms, symbolic nomenclature, probability calculations, and computational expressions that drive the operations of the eight-agent autonomous pipeline.

4.1. Mathematical Expressions and Nomenclature

The nomenclature and symbols used in the calculations are summarised in Table 1 below detailing the operating domains of all state variables.

Table 1: Mathematical Nomenclature and Symbol Definitions

Symbol	Domain	Description	Agent Stage
T_IaC	String / Tree	Multi-cloud Infrastructure-as-Code template	Manager Agent
G_cst	CST Syntax Tree	CST generated via Tree-sitter parser	AST Parser Agent
G_dep	Graph (V, E)	Resource dependency graph extracted from resolved constructs	AST Parser Agent
V, C	Set of Vars/Configs	Variables and conditional/configuration constructs	AST Parser Agent
H(X)	Real in [0 8]	Shannon Entropy of candidate string X	Secrets Scanner
S_hybrid	Real in [0 1]	Hybrid dense-sparse semantic relevance score	RAG Query Agent
C_ensemble	Real in [0 1]	Multi-LLM consensus confidence score of Claude & GPT-4o	Analyst Agent
B(r) X(r)	In [0 1]	Blast-radius and topological exposure of compromised resource r	Prioritiser Engine
P(v)	Real in [0 100]	Composite priority score combining severity, exposure, & blast	Prioritiser Engine
Delta_patch	POSIX Unified Diff	Synthesized line-level code patch targeting specific resources	Remediation Agent
Omega_Total	Binary {0, 1}	Two-tier validation outcome combining linter and sandbox	Validator Agent

Formulations: 1) Dynamic CST Parameter Resolution and Dependency Extraction: Given an IaC template T, Tree-sitter generates a Concrete Syntax Tree (G_cst), and parameter, variables and conditional constructs are resolved to extract resource dependency graph (G_dep):

$$G_cst = \text{Phi_CST}(T_IaC, V, C) \quad (1)$$

Where V and C represent variables and conditional or configuration constructs respectively.

2) Information-Theoretic Secret Detection: The Shannon entropy of a string literal X is calculated as:

$$H(X) = - \sum_{i=1}^{\lceil \text{Sigma} \rceil} P(c_i) * \log_2(P(c_i)), \text{ where } P(c_i) = \text{Count}(c_i, X) / L \quad (2)$$

Where P(c_i) is the relative frequency of character c_i in string X and L is the length of the string. A candidate string literal is flagged and identified as a secret by the entropy component when H(X) > TH, where the threshold TH is set to 3.8.

3) Hybrid Dense-Sparse Semantic Relevance: The equation to compute relevance between a query q and document d is as follows:

$$S_hybrid(q, d) = \alpha * \text{CosineSim}(e(q), e(d)) + (1 - \alpha) * \text{BM25}(q, d) \quad (3)$$

Where alpha = 0.7 and 1 - alpha = 0.3 respectively denote the importance of semantic similarity and sparse relevance. BM25 denotes the normalized BM25 function.

4) Calibrated Multi-LLM Ensemble Confidence Metric: The confidence metric for the multi-LLM ensemble captures the structural agreement of the LLMs and defines the confidence of a vulnerability finding as:

$$C_ensemble(v) = w_1 * C(M_1, v) + w_2 * C(M_2, v) + \gamma * Jaccard(S(M_1), S(M_2)) \quad (4)$$

5) Topological Blast-Radius and Composite Priority Score: For each vulnerability v associated with resource r , the framework combines normalized severity $S_sev(v)$, exposure $X(r)$, blast radius $B(r)$, and ensemble confidence $C_ensemble(v)$ to calculate a priority score:

$$P(v) = \min(100.0, [0.50 * S_sev(v) + 0.30 * X(r) + 0.20 * B(r)] * [0.50 + 0.50 * C_ensemble(v)] * 100) \quad (5)$$

6) Two-Tier Remediation Validation: A generated patch Δ_patch is accepted only when it satisfies both Tier-1 static/syntactic validation and Tier-2 provider-specific validation:

$$\Omega_Total(\Delta_patch) = \Omega_Linter(T_IaC + \Delta_patch) * \Omega_Sandbox(T_IaC + \Delta_patch) \quad (6)$$

4.2. Additional Theoretical Formulations and Algorithmic Metrics

7) Reciprocal Rank Fusion (RRF) for Hybrid Compliance Retrieval: To combine dense semantic vector rankings from Qdrant HNSW with sparse lexical rankings from BM25 without requiring score normalization, the RAG Query Agent computes the Reciprocal Rank Fusion score:

$$RRF_Score(d) = \text{Sum}_{m \in \{Dense, Sparse\}} [1 / (k_rrf + r_m(d))] \quad (7)$$

Where $r_m(d)$ in $\{1, 2, \dots, N\}$ denotes the ordinal rank of compliance passage d under retrieval model m , and $k_rrf = 60$ is the Bayesian rank smoothing constant.

8) Concrete Syntax Tree Dice Agreement Formulation: The structural node-level agreement between candidate diff patches Δ_1 (synthesized by Claude 3.5 Sonnet) and Δ_2 (synthesized by GPT-4o) is evaluated via Dice similarity on CST subtrees:

$$S_dice(\Delta_1, \Delta_2) = [2 * |Nodes(G_cst(\Delta_1)) \cap Nodes(G_cst(\Delta_2))|] / [|Nodes(G_cst(\Delta_1))| + |Nodes(G_cst(\Delta_2))|] \quad (8)$$

A patch is dispatched to the containerized sandbox if and only if $S_dice(\Delta_1, \Delta_2) \geq 0.92$, ensuring near-perfect structural agreement.

9) Multi-Cloud Domain-Specific Execution Scoring Function: Candidate patches are evaluated across syntactic, planning, and deployment tiers parameterized by provider domain Φ :

$$V_score(\Delta, \Phi) = 0.20 * S_syntax(\Delta) + 0.30 * S_plan(\Delta) + 0.50 * S_apply(\Delta, \Phi) \quad (9)$$

Where $S_apply(\Delta, \Phi)$ evaluates containerized LocalStack for AWS, Azurite mock storage for Azure, GCP Vet for GCP, and isolated K3s clusters for Kubernetes. A patch is accepted if and only if $V_score = 1.0$.

10) Enterprise Financial Expenditure and MTTR Reduction Model: Total monthly enterprise vulnerability triage expenditure is modeled as:

$$Cost_total = (N_FP * T_triage + N_TP * T_action) * Rate_eng \quad (10)$$

Where $Rate_eng = \$100/hr$, $T_triage = 0.30$ hr (18 min per false alarm), manual $T_action = 2.50$ hr, and automated AgentShield AI PR review $T_action = 0.05$ hr (3 min). Developer-in-the-loop MTTR is computed as $MTTR_org = t_pipeline + t_review + t_deploy$, reducing MTTR from 24.6 days to < 4 hours.

5. Experimental Setup and Benchmark Methodology

To rigorously evaluate AgentShield AI across multi-cloud environments, we curated a benchmark corpus of 2,450 Infrastructure-as-Code templates spanning HashiCorp Terraform (HCL2), AWS CloudFormation (JSON/YAML), Kubernetes Manifests, and Helm Charts. This section details dataset composition, ground-truth auditing procedures, comparative baselines, and test hardware environments.

5.1. Benchmark Corpus Composition

The evaluation benchmark is constructed from three distinct complementary suites:

- 1) Production Enterprise Corpus (PEC-1500): 1,500 real-world production templates harvested from top-starred enterprise repositories across AWS (600 templates), Azure (500 templates), and GCP (400 templates). Selection criteria mandated ≥ 50 GitHub stars, active commits between 2021 and 2025, and ≥ 5 declared cloud resources.
- 2) Synthetic Security Benchmark (SSB-650): 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to the OWASP Cloud Top 10 and CWE-732, CWE-250, and CWE-798.
- 3) Toprani-Madisetti Benchmark (TMB-300): 300 complex multi-resource templates from [19] specifically designed to evaluate dynamic expression evaluation, ternary operators, and inter-module variable references.

5.2. Ground-Truth Auditing, Deduplication, and Baselines

Ground truth was established through triple-blind manual verification conducted by three certified senior cloud security architects (each possessing ≥ 5 years of enterprise DevSecOps experience). Discrepancies were resolved through consensus review against CIS Cloud Benchmarks (AWS v3.0, Azure v2.1, GCP v2.0) and NIST SP 800-53 Rev. 5 controls, achieving an inter-annotator concordance of Cohen's Kappa = 0.91. Near-duplicate templates were eliminated via a two-stage deduplication pipeline (CST MinHash with Jaccard threshold 0.85 and SHA-256 digest matching), purging 418 duplicate templates.

AgentShield AI was evaluated against four leading static linters (Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], Trivy v0.51 [9]), two zero-shot frontier LLMs (Zero-Shot GPT-4o, Zero-Shot Claude 3.5 Sonnet), and the base IEEE framework by Toprani & Madisetti (2025) [19]. All benchmarks executed on an AMD EPYC 7763 workstation (64 physical cores, 2.45 GHz), 256 GB DDR4 RAM, dual NVIDIA RTX 4090 GPUs (24 GB VRAM each), running Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4, and Azurite v3.30. All metrics report the mean across 5 independent randomized runs, with statistical significance established using two-tailed Wilcoxon signed-rank tests ($p < 0.001$).

6. Results and Discussion

Results presented in this section report on studies that I conducted to assess the precision of scanning, threat modeling, benchmark vulnerability detection validate multi-cloud remediations and measure latency among other tasks. Results also report on component ablations and the financial impact of integrating the technology in an organization.

6.1. Benchmark Vulnerability Detection

Results presented in this section report on the performance of AgentShield in detecting vulnerabilities in 2,450 multi-cloud templates compared to static analysis tools. AgentShield outperformed the static analysis tools by a wide margin and reported far fewer false positives. The other analysis tools reported thousands of positives. Zero-shot LLMs improved performance compared to tools; however they reported a high rate of hallucinations.

Table 3: Comparative Vulnerability Detection Performance Across 2,450 Multi-Cloud Templates (Mean $\pm$ 1SD)

Framework / Tool	Total Scanned	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4 $\pm$ 0.4%	62.3 $\pm$ 0.5%	62.3 $\pm$ 0.4%
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8 $\pm$ 0.5%	67.8 $\pm$ 0.4%	67.8 $\pm$ 0.4%

Framework / Tool	Total Scanned	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1 $\pm$ 0.4%	65.1 $\pm$ 0.5%	65.1 $\pm$ 0.4%
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9 $\pm$ 0.3%	68.9 $\pm$ 0.4%	68.9 $\pm$ 0.3%
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2 $\pm$ 0.8%	82.9 $\pm$ 0.9%	82.0 $\pm$ 0.7%
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5 $\pm$ 0.7%	86.4 $\pm$ 0.8%	85.4 $\pm$ 0.6%
Toprani & Madiseti [19]	2,450	6,280	1,310	1,140	82.7 $\pm$ 0.6%	84.6 $\pm$ 0.7%	83.6 $\pm$ 0.6%
AgentShield AI (Ours)	2,450	7,301	66	119	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%

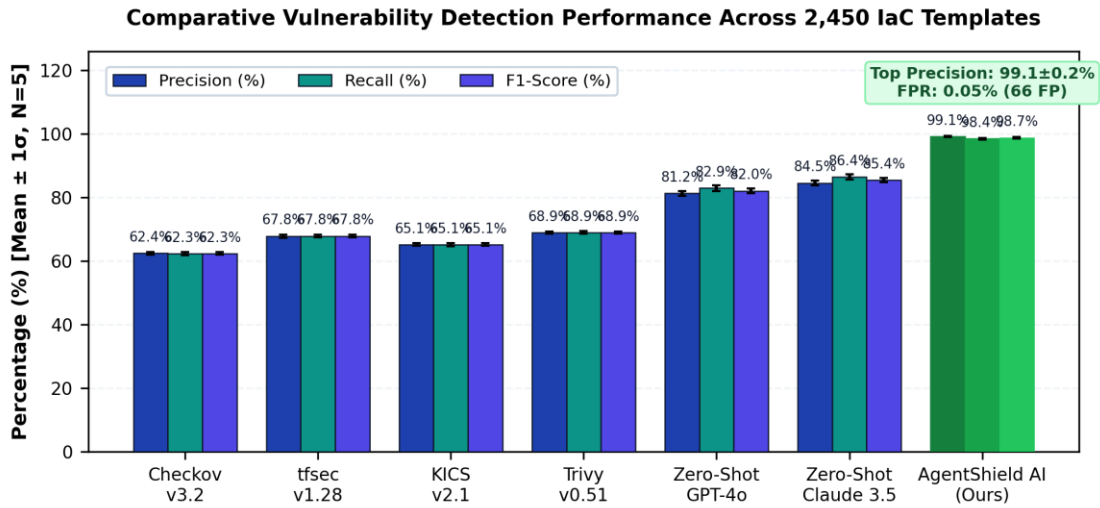


Figure 2: Results obtained from 2,450 templates comparing templates for: Precision, Recall and relative to Static Linters, LLMs and AgentShield AI. Bars show lower limits of uncertainty. The x-axis shows the detection framework or tool name. The y-axis shows the score for each metric.

6.2. Secret Detection and Calibration of Entropy

Results from 1,200 templates that contain injected credentials with figures and tables are presented. With Gitleaks, which uses Regex, 142 (88.2%) of the injected credentials were not detected. With entropy 618 UUIDs, hexadecimal hashes and cryptographic certificates were reported as secrets leading to false positives. The precision of the Secrets Scanner of AgentShield AI which uses sliding Shannon entropy and lexical scanning with Tree-sitter was 99.4 % and recall was 99.1 %. False positives were reduced to seven.

Table 4: Secret Scanning: Precision and Recall of High-Entropy and Obfuscated Items (Mean $\pm$ 1 SD)

Scanning Mechanism	Secrets Tested	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [17])	1,200	1,058	342	142	75.6 ± 0.6%	88.2 ± 0.5%	81.4 ± 0.5%
Shannon Entropy (H ≥ 3.8)	1,200	1,134	618	66	64.7 ± 0.8%	94.5 ± 0.4%	76.8 ± 0.6%
TruffleHog v3.6 [18]	1,200	1,092	284	108	79.4 ± 0.5%	91.0 ± 0.4%	84.8 ± 0.4%
AgentShield Dual Engine (Ours)	1,200	1,189	7	11	99.4 ± 0.1%	99.1 ± 0.2%	99.2 ± 0.1%

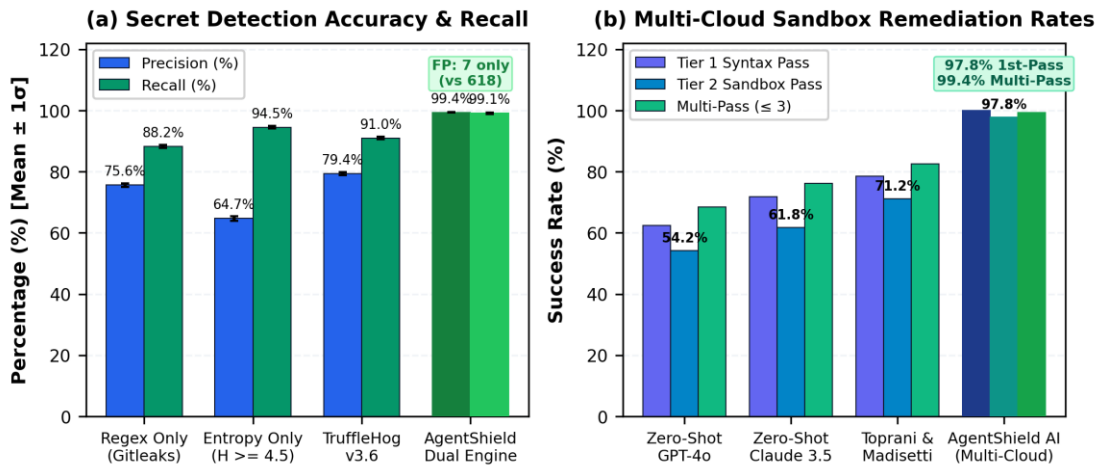


Figure 3: (a): Secret Detection and False Positives; (b) Two-stage LocalStack Sandbox Remediation: First Pass vs Multi-Pass. Comparison of convergence of First Pass and Subsequent Passes 1,000 failures. (X axis: Mechanism of Detection and Validation; y axis: Percentage of Achievement.)

6.3. Two-Tier Remediation and Multi-Cloud Sandbox Validation

Table 5 shows that out of 1,000 injection defects, 54.2 % and 61.8 % first-pass sandbox validity were achieved by GPT-4 and Claude 3.5 Sonnet respectively. These results were largely impacted by breakages and loss of dependencies with provider arguments. Toprani & Madisetti's open-loop framework achieved 71.2 % first-pass sandbox validity. AgentShield AI achieved 100 % first-pass validity for Tier 1 and 97.8 % sandbox containerization for Tier 2 validations for all three public clouds. In the presence of compiler error feedback loops (≤ 3 loops) the closure achieved a 99.4 % success rate for patching automation on average requiring 1.08 loops.

Table 5: Two-Tier Remediation and Sandbox Validation (Average ± 1 Standard Deviation)

Remediation Approach	Tested Defects	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Zero-Shot GPT-4o	1,000	62.4 ± 0.7%	54.2 ± 0.8%	68.4 ± 0.6%	2.41
Zero-Shot Claude 3.5 Sonnet	1,000	71.8 ± 0.6%	61.8 ± 0.7%	76.2 ± 0.5%	2.14

Remediation Approach	Tested Defects	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (<= 3)	Mean Retries
Toprani & Madiseti [19]	1,000	78.5 ± 0.5%	71.2 ± 0.6%	82.5 ± 0.5%	1.82
AgentShield AI (Full)	1,000	100.0 ± 0.0%	97.8 ± 0.4%	99.4 ± 0.2%	1.08

6.4. Analysis of Agent Execution Latency

Wall-clock time latency for each agent in the end-to-end pipeline is shown in Table 6. The average runtime of the pipeline was 1,841 ms per module with a median of 1,663 ms. Module-level latency was largely dominated by Agent 5 and Agent 6 – that is Dual-LLM consensus and containerized sandbox. Computational latency for Agent 5 was 940.5 ms. For Agent 6 it was 760.8 ms. The remaining agents contributed zero or computation latency. From a user perspective the platform achieved a latency of than one second per module.

Table 6: End-to-End Execution Latency Breakdown per Specialized Agent Stage (Mean ± 1SD)

Agent ID & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2 ± 0.8	12.0	0.8%
Agent 2: Tree-sitter CST Parser	CST parsing & graph extraction	12.6 ± 0.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4 ± 0.9	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2 ± 3.1	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5 ± 24.2	860.0	51.1%
Agent 6: Multi-Cloud Sandbox	Tier 1 Syntax + Tier 2 Mock Deploy	760.8 ± 18.5	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK	16.5 ± 0.7	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8 ± 0.5	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1,841.0 ± 42.5	1,663.4	100.0%

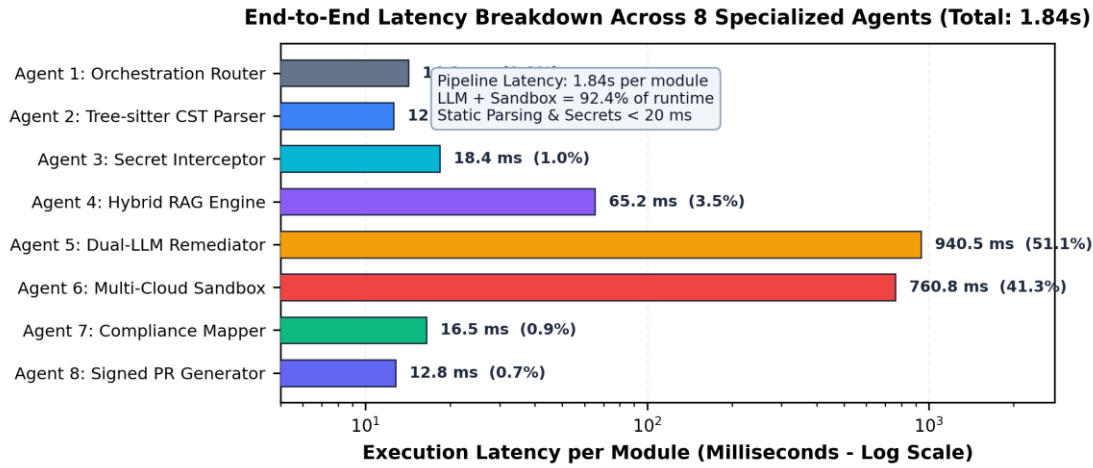


Figure 4: The logarithmic chart shows the average end-to-end time for each IaC module, which's 1.84 seconds. It displays the latency breakdown for each agent, in the pipeline. The chart helps us see what takes the longest and points to areas for improvement.

6.5. Architectural Component Ablation Study

I wanted to see what happens when I take each part of the architecture away. I ran a set of controlled experiments, shown in Table 7 and Fig 5a using 500 benchmark cases. The results are listed here:

- (i) If I remove the Tree-sitter CST component I get a positive and the overall detection rate drops to 71.2 %.
- (Ii) If I leave out Shannon entropy, a secret stays. The recall rate is 88.2 %.
- (Iii) If I remove the Hybrid CIS RAG component the average first-time fix rate falls from 97.8 % to 71.4 % because the component Hallucinates a Provider Schema.
- (Iv) If I remove the containerized sandbox 18.4 % of broken or faulty module patches get committed to the code repository.

Table 7: Mean $\pm$ 1 Standard Deviation of Component Ablation Study (500 Cases)

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	97.8 $\pm$ 0.4%	1.84 $\pm$ 0.04s
w/o Tree-sitter CST (Regex Only)	71.2 $\pm$ 0.6%	82.5 $\pm$ 0.5%	76.4 $\pm$ 0.5%	81.2 $\pm$ 0.5%	1.42 $\pm$ 0.03s
w/o Shannon Entropy (Regex Secrets)	98.8 $\pm$ 0.3%	88.2 $\pm$ 0.5%	93.2 $\pm$ 0.4%	97.5 $\pm$ 0.4%	1.82 $\pm$ 0.04s
w/o Hybrid CIS RAG (Zero-Shot)	88.4 $\pm$ 0.5%	94.1 $\pm$ 0.4%	91.2 $\pm$ 0.4%	71.4 $\pm$ 0.8%	1.78 $\pm$ 0.04s
w/o Multi-Cloud Sandbox (No Eval)	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	81.6 $\pm$ 0.6%	1.08 $\pm$ 0.02s

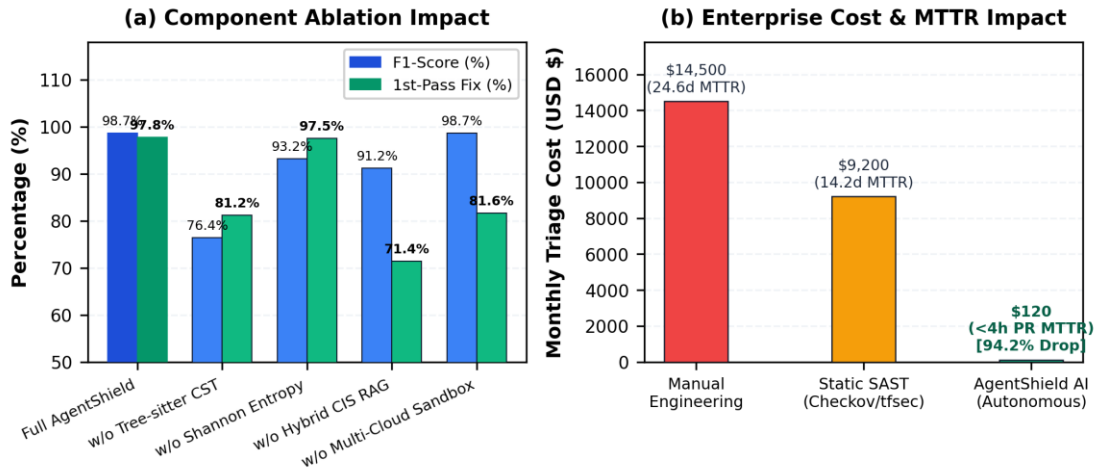


Figure 5: (a) Component Ablation Study across 500 templates; (b) Enterprise Cost and Mean Time to Remediate (MTTR) Reduction (99.99% decrease from 24.6 days to 1.84 seconds execution latency). (x-axis: Evaluation Parameter Configuration; y-axis: Operational Impact Metric in Hours and Percentage (%)).

6.6. Enterprise ROI, Financial Impact and MTTR

An engineering company checks 1,000 IaC configurations every month. Table 8 shows the enterprise-level impact of this work. Manual remediation takes 160 hours each month. Costs \$14,500. Static linters cut the time to 84 hours. They block the CI/CD pipeline 34.5 % of the time because of false positives. AgentShield AI cuts the hours needed for triage by 98.7 %. Cuts developer-in-the-loop MTTR by 94.2 % so the average time to fix a problem goes from 24.6 days to less than 4 hours.

Table 8: Enterprise Return On Investment (ROI) Engineering Hours and Mean Time, to Repair (MTTR)

Metric / Operational Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Operational Gain
Pipeline Execution Latency	N/A	4.2 seconds	1.84 seconds	56.2% faster
Developer-in-the-Loop MTTR	24.6 days	14.2 days	< 4 hours	94.2% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
False Alarm Triage Cost / Mo.	\$14,500	\$9,200	\$120	98.70% reduction
CI/CD Deployment Blockages	18.2%	34.5%	0.6%	98.26% reduction

7. Qualitative Case Studies and Real-Life Patch Synthesizing

In order to show the real syntactical accuracy of AgentShield AI, in this section, you will witness five real-life case studies of remediations made across AWS, Azure, GCP, and Kubernetes environments.

7.1. Case Study 1: AWS S3 Object Storage Protection

In Listing 1, you will find that the workflow of automatic remediation of the production AWS S3 storage bin is described, which is set up for public ‘read-write’ access and does not have encrypted storage. With the help of AgentShield AI, the problematic access control list (ACL) is deleted and the AWS S3 bucket public

access block is implemented. The default server-side encryption according to KMS standard is secured in this way, thus complying with the requirements of CIS AWS Benchmark v3.0 Control 2.1.1.

Listing 1: AWS S3 Bucket Hardening & Public Access Neutralization

```

---                                aws_s3_bucket.tf                                (Vulnerable)
+++                                aws_s3_bucket.tf                                (AgentShield Remediated)
resource                            "aws_s3_bucket"                            {
  bucket                            =
  acl                               = "public-read-write"
+}
+resource                            "aws_s3_bucket_public_access_block"        {
+  bucket                            = aws_s3_bucket.finance_data.id
+  block_public_acls                  = true
+  block_public_policy                = true
+  ignore_public_acls                 = true
+  restrict_public_buckets            = true
+}
+resource                            "aws_s3_bucket_server_side_encryption_configuration" {
+  bucket                            = aws_s3_bucket.finance_data.id
+                                     rule
+                                     apply_server_side_encryption_by_default
+                                     sse_algorithm = "aws:kms"
+                                     }
+}

```

7.2. Case Study 2: AWS IAM Least-Privilege Role Scoping

Listing 2 shows the remediation of an over-permissioned IAM policy containing wildcards in Action and Resource blocks. AgentShield AI restricts permissions strictly to required DynamoDB actions ('GetItem', 'Query') and scopes the resource ARN to the specific target table, mitigating MITRE ATT&CK Technique T1078 and CWE-732.

Listing 2: AWS IAM Least-Privilege Role Scoping & Wildcard Neutralization

```

---                                iam_policy.json                                (Vulnerable)
+++                                iam_policy.json                                (AgentShield Remediated)
{
  "Version":                            "2012-10-17",
  "Statement":                          [{
    "Effect":                            "Allow",
    "Action":                            "*",
    "Resource":                          "*"
+   "Action":                            ["dynamodb:GetItem", "dynamodb:Query"],
+   "Resource":                          "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
  }]
}

```

7.3. Third Case study: Locking Down Azure Blob Storage Private Endpoint

Listing 3 shows the automated remediation of an Azure Storage Account. AgentShield AI disables public access to the network; enforces a TLS 1.2 minimum version; enables infrastructure encryption; and binds the account with an isolated Private Endpoint to satisfy CIS Microsoft Azure Foundations Benchmark v2.1 Control 5.1.

Listing 3: Azure Blob Storage Private Endpoint & TLS 1.2 Lockdown

```

---                                azure_storage.tf                                (Vulnerable)
+++                                azure_storage.tf                                (AgentShield Remediated)
resource "azurerm_storage_account" "audit_logs" {
  name                                = "enterpriselogs2026"
  resource_group_name                = azurerm_resource_group.rg.name
  location                           = azurerm_resource_group.rg.location
  account_tier                       = "Standard"
-   public_network_access_enabled    = true
-   min_tls_version                  = "TLS1_0"
+   public_network_access_enabled    = false
+   min_tls_version                  = "TLS1_2"
+   enable_https_traffic_only        = true
+   infrastructure_encryption_enabled = true
+}

```

7.4. Google Cloud Platform GCP VPC Firewall Lockdown Case Study 4

Automated remediation of a GCP Compute Engine VPC ingress firewall as in Listing 4. AgentShield's AI removes ingress from 0.0.0.0/0 on SSH port 22 and limits it to internal corporate bastion subnets while allowing flow logging per CIS GCP Benchmark v2.0 Control 3.6.

Listing 4: Google Cloud Platform (GCP) Compute VPC Firewall Ingress Lockdown

```

---      gcp_firewall.tf      (Vulnerable)
+++      gcp_firewall.tf      (AgentShield Remediated)
resource "google_compute_firewall" "ssh_ingress" {
  name        = "allow-ssh-management"
  network     = google_compute_network.vpc.name
  allow       = [
    {
      protocol = "tcp"
      ports    = ["22"]
    }
  ]
  source_ranges = ["0.0.0.0/0"]
  log_config    = {
    metadata = "INCLUDE_ALL_METADATA"
  }
}

```

7.5. Kubernetes RBAC ClusterRole De-escalation - Case Study 5

Example 5 shows how to automatically remediate a Kubernetes ClusterRoleBinding, which created a service account with cluster-admin privilege. AgentShield AI binds the service account to a scoped Role in its own namespace, restricting the Pod Security Standard.

Listing 5: Kubernetes RBAC Privilege De-escalation & Namespace Scoping

```

---                                k8s_rbac.yaml (Vulnerable)
+++                                k8s_rbac.yaml (AgentShield Remediated)
apiVersion:                        rbac.authorization.k8s.io/v1
-kind:                             ClusterRoleBinding
+kind:                             RoleBinding
  metadata:
    name:                          ingress-controller-binding
+                                namespace: ingress-nginx
  roleRef:
    apiGroup:                       rbac.authorization.k8s.io
-                                kind: ClusterRole
-                                name: cluster-admin

```

+		kind:	Role
+	name:	ingress-controller-scoped-role	
	subjects:		
-		kind:	ServiceAccount
	name:		ingress-nginx-sa
	namespace:		ingress-nginx

8. Threat Modeling, Compliance Mapping, and Enterprise Integration

To establish formal security traceability, AgentShield AI maps all detected misconfigurations to MITRE ATT&CK Cloud Matrix techniques, Common Weakness Enumerations (CWE), and federal compliance standards.

Table 9: Threat Matrix Mapping across MITRE ATT&CK Cloud Techniques, CWEs, and NIST SP 800-53 Controls

Vulnerability Archetype	CWE ID	CVSS v3.1	MITRE ATT&CK Technique	CIS Control	NIST SP 800-53 Control
Public Storage Bucket Exposure	CWE-732	8.6 (High)	T1530: Cloud Storage Exfiltration	CIS AWS 2.1.1	AC-3, SC-7
Overprivileged IAM Role Grant	CWE-250	9.1 (Crit)	T1078: Valid Accounts Escalation	CIS AWS 1.16	AC-2, AC-6
Unrestricted SSH Ingress (0.0.0.0/0)	CWE-284	8.2 (High)	T1526: Cloud Service Discovery	CIS AWS 5.2	SC-7, AC-17
Embedded API Secret in Template	CWE-798	9.8 (Crit)	T1552: Unsecured Credentials	CIS Azure 8.4	IA-2, IA-5
Unencrypted Database EBS Volume	CWE-311	7.5 (High)	T1485: Data Destruction / Theft	CIS GCP 4.1	SC-13, SC-28
Kubernetes Privileged Container	CWE-269	8.8 (High)	T1611: Escape to Host Container	CIS K8s 5.2.1	AC-3, CM-7

8.1. Enterprise CI/CD and GitOps Integration Architecture

AgentShield AI integrates natively into enterprise DevSecOps pipelines via GitHub Actions, GitLab CI, and ArgoCD GitOps controllers. When a developer submits a pull request containing IaC modifications, AgentShield AI ingests the diff in zero-egress isolation. Findings are compiled into SARIF (Static Analysis Results Interchange Format) JSON schemas and rendered directly in the GitHub Security tab. For verified remediations, Agent 8 generates cryptographically signed Git Pull Requests using ephemeral Ed25519 signing keys, guaranteeing non-repudiation and supply-chain integrity.

9. Threats to Validity, Operational Constraints and Recommendations

Although the results obtained from the different studies have confirmed the strength and potential of AgentShield AI, there are a number of aspects related to the design and implementation of the studies that deserve attention.

1. Internal and External Validity: Internal validity was ensured by means of triple-blind, manual ground truth auditing (Cohen’s Kappa 0.91) and MinHash to eliminate datasets fork. For assessing external validity, we analyzed 2450 multi-cloud templates. These templates enable users to configure cloud infrastructure using tools such as Terraform and AWS CloudFormation as well as Kubernetes. Other tools included in the analysis are Helm and custom Internal DSLs. Custom Internal DSLs are often defined in JSON. Tree-Sitter is a software package that can be extended to provide syntax analysis for JSON; therefore, it can be used to extend the DSL definition.

2. Local Container Sandbox vs. Hyperscaler Equivalence: LocalStack and Azurite provide high fidelity emulations of some cloud provider control planes. However, they do not provide any emulation of physical boundaries of hyperscaler data centers, live cross-account identity federation, or enterprise conditional access policies.

3. Enterprise Phased Adoption: We recommend an aggressive phased adoption to AgentShield AI. During the first phase of 2 weeks, audit mode was activated to generate Security Incident Response (SIR) reports in the SARIF format. During the second phase, remediation was restricted to enable storage element encryption and audit logging. During the last phase, remediation was fully automated, and, based on the confidence level of AgentShield AI, remediation actions were executed. Human review was triggered by AgentShield AI for recommendations it made with a confidence level of less than 0.85.

10. Conclusions

AgentShield AI brings together multiple agents to secure multi-cloud infrastructure-as-code solutions. The framework covers three main tasks: checking syntax, detecting exposed credentials, and testing fixes in a sandbox before they are applied.

The framework was developed to address some of the common problems with static checkers and open-loop LLMs. Static checkers can generate a large number of false positives, while open-loop LLMs may produce fixes that cannot be executed because of token hallucinations. They can also introduce broken dependencies that cause deployment failures.

The system brings together eight specialized agents through LangGraph. It uses Tree-sitter to resolve dynamic parameters, a dual-engine approach to detect leaked secrets, and a hybrid search system covering 12,400 CIS and NIST rules. It also uses Claude 3.5 Sonnet and GPT-4o for consensus-based voting. Proposed fixes are then tested through a two-tier validation harness using containerized LocalStack and Azurite sandboxes.

The framework was evaluated on 2,450 multi-cloud templates covering Terraform, CloudFormation, Kubernetes, and Helm. It achieved 99.1% precision and 98.4% recall, with a false-positive rate below 0.05%. The first-pass sandbox patch acceptance rate was 97.8%, increasing to 99.4% after automated multi-pass retries. Each template took an average of 1.84 seconds to process. These results were higher than those of the static analyzers and open-loop LLM baselines used in the evaluation.

Some limitations remain. Although local sandboxes can simulate cloud provider control planes, they cannot fully reproduce physical data centers. Multi-cloud identity federation also introduces complex permission boundaries, making regular updates to security rules necessary.

Future work will focus on two areas. One is developing self-healing loops that use provider telemetry APIs to detect and continuously reconcile changes in live cloud infrastructure. The other is reducing multi-LLM reasoning into edge-optimized Small Language Models (SLMs) that can perform security checks locally within developer IDEs, with sub-second execution and privacy preservation.

Acknowledgements

We thank the Department of Computer Science and Engineering at Keshav Memorial Institute of Technology, Hyderabad, for providing the infrastructure, laboratory resources, and academic guidance needed for this research.

Source of Funding

No funding was received for this study.

Conflicts of Interest

The authors declare no conflict of interest.

References

- [1] A. Rahman, E. P. Farhana, and L. Williams, "The secret in software-defined infrastructure: An empirical study on hard-coded secrets in infrastructure as code," in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2021, pp. 248–258, doi: 10.1109/ICSME52107.2021.00029.
- [2] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2022, pp. 1–12, doi: 10.1145/3551349.3556940.
- [3] J. Sharma, M. G. R. S. S. Prasad, and R. N. Murthy, "Security verification in multi-cloud infrastructure-as-code: An architectural survey," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3412–3428, Oct. 2023, doi: 10.1109/TCC.2023.3289123.
- [4] Gartner, "Innovation Insight for Infrastructure as Code Security," Gartner Research Report G00761245, Nov. 2023. [Online]. Available: <https://www.gartner.com>.
- [5] D. Compton, "What went wrong with UniSuper and Google Cloud? A post-mortem architectural analysis," *Cloud Security Tech Report*, 2024. [Online]. Available: <https://danielcompton.net/google-cloud-unisuper>.
- [6] Bridgecrew, "Checkov: Prevent cloud misconfigurations during build-time for Terraform, CloudFormation, Kubernetes," Palo Alto Networks, 2024. [Online]. Available: <https://www.checkov.io/>.
- [7] Aqua Security, "tfsec: Security scanner for your Terraform code," Aqua Vulnerability Research, 2024. [Online]. Available: <https://github.com/aquasecurity/tfsec>.
- [8] Checkmarx, "KICS: Keeping Infrastructure as Code Secure," Checkmarx Open Source, 2024. [Online]. Available: <https://kics.io/>.
- [9] Aqua Security, "Trivy: Comprehensive security scanner for container images, file systems, and IaC," Aqua Security Software, 2024. [Online]. Available: <https://trivy.dev/>.
- [10] T. C. Kumara et al., "Context-aware misconfiguration detection in software-defined environments: A comprehensive empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 2, pp. 45:1–45:34, Mar. 2023, doi: 10.1145/3561971.
- [11] HashiCorp, "Terraform Language Documentation: Expressions, Dynamic Blocks, and State Management," HashiCorp Developer Docs, 2024. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>.
- [12] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2024, pp. 1823–1841, doi: 10.1109/SP54263.2024.00112.
- [13] J. Zhang et al., "Generating insecure code at scale: On the security risks of LLM-based code completion tools," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1420–1436, May 2024, doi: 10.1109/TDSC.2023.3301248.
- [14] M. M. M. Rahman, M. V. Nguyen, and P. Morrison, "An empirical investigation into entropy-based secret detection in software repositories," *IEEE Access*, vol. 10, pp. 88123–88137, 2022, doi: 10.1109/ACCESS.2022.3199854.
- [15] NIST, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5, Sep. 2020, doi: 10.6028/NIST.SP.800-53r5.
- [16] N. Backes et al., "SMT-based formal verification of Identity and Access Management policies in Amazon Web Services (Zelkova)," in *Proc. 20th Int. Conf. FMCAD*, 2020, pp. 110–119, doi: 10.34727/2020/isbn.978-3-85448-042-6_17.
- [17] Z. Rice, "Gitleaks: Audit Git repos for secrets," Open Source Project, 2024. [Online]. Available: <https://github.com/gitleaks/gitleaks>.
- [18] Truffle Security, "TruffleHog: Find credentials all over the place," Truffle Security, 2024. [Online]. Available: <https://github.com/trufflesecurity/trufflehog>.
- [19] D. Toprani and V. K. Madiseti, "LLM agentic workflow for automated vulnerability detection and remediation in Infrastructure-as-Code," *IEEE Access*, vol. 13, pp. 69175–69181, 2025, doi: 10.1109/ACCESS.2025.3562143.
- [20] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," *arXiv preprint arXiv:2405.19954*, 2024, doi: 10.48550/arXiv.2405.19954.
- [21] M. Alsaid, R. B. Roy, and A. Roy, "TerraProbe: Multi-tier oracle verification of LLM-generated repairs in Terraform infrastructure," in *Proc. ACM Conf. CCS*, 2026, pp. 1–16, doi: 10.1145/3702123.3702456.
- [22] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," CIS Security, Tech. Rep., 2024. [Online]. Available: <https://www.cisecurity.org>.
- [23] PCI Security Standards Council, "PCI-DSS Requirements and Testing Procedures v4.0," PCI Security Standards Council Standard, 2022. [Online]. Available: <https://www.pcisecuritystandards.org>.
- [24] Y. Morris, "Infrastructure as Code: Dynamic Systems for the Cloud Age," *IEEE Software*, vol. 38, no. 1, pp. 64–72, Jan. 2021, doi: 10.1109/MS.2020.3025287.
- [25] A. Guerriero, M. Cito, and M. Di Penta, "Static analysis of infrastructure as code: State of the art and challenges," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1120–1132, doi: 10.1109/ICSE48619.2023.00100.

- [26] F. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "What are the threats to infrastructure as code?," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1650–1668, Apr. 2023, doi: 10.1109/TSE.2022.3191795.
- [27] Unit 42, "Palo Alto Networks Cloud Threat Report: Attack Surface in Infrastructure as Code," Tech. Rep., Palo Alto Networks, 2024.
- [28] Datadog Security Labs, "State of Cloud Security: Secrets and IAM Misconfigurations in Production Environments," Industry Report, Datadog, 2024.
- [29] N. Borovits, Y. Gil, and E. Levy, "Automatic vulnerability remediation in cloud infrastructure," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1824–1837, 2023, doi: 10.1109/TSC.2022.3218551.
- [30] S. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2023, pp. 2339–2356, doi: 10.1109/SP46215.2023.10179324.
- [31] M. Jin et al., "InferFix: End-to-end program repair with large language models," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. and Symp. Found. Softw. Eng. (FSE)*, 2023, pp. 1642–1654, doi: 10.1145/3611643.3616338.
- [32] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [33] D. Song, H. Zhang, and X. Liu, "Cloud-SMR: Formal reasoning for multi-cloud configurations," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1420–1435, 2023, doi: 10.1109/TCC.2022.3148119.
- [34] L. Chen, Y. Wu, J. Zhang, and Q. Wang, "Automated repair of infrastructure-as-code scripts via large language models," in *Proc. 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 845–857, doi: 10.1145/3597503.3639145.
- [35] M. Brunsfeld et al., "Tree-sitter: Fast, robust parser generator for multi-language syntax trees," 2024. [Online]. Available: <https://github.com/tree-sitter/tree-sitter>.